

Abstract. We report automated heuristic verification for three configurations, always listed in evaluation order ① **OpenCode + RAG** (repository roots, Pinecone retrieval, five checks including control-grounding), ② **GPT direct** (single-file, four checks, OpenAI gpt-5.4-mini in this dataset), ③ **OpenCode baseline** (agent, four checks, no retrieval). The headline mean for ② (~0.905 on [0,1]) is *not* “94% accuracy”; it is the mean fraction of checks passed under a four-item rubric. RAG uses a stricter five-item rubric and larger, truncated inputs. Section 2 quantifies ingestion; Section 6 explains confounders (model identity, input cardinality, rubric size).

1. Experimental design

1.1 Configurations (fixed ordering)

- ① **OpenCode + RAG** — `mode: rag`; agent loop; top-5 ISO control chunks from Pinecone; verification includes *control-grounding* (cited control IDs must appear in retrieved set). Targets: *repository roots* under `samples/bench50/*`, text ingested with a hard character ceiling.
- ② **GPT direct** — `mode: direct`; single completion; no retrieval corpus; four verification checks. Targets: *single files* (same bench50 corpus family, one path per run).
- ③ **OpenCode baseline** — `mode: baseline`; OpenCode agent with file attachment; four checks; no vector retrieval.

1.2 Verification rubric (implementation)

All checks are implemented in `packages/opencode/src/security/verification.ts` and operate on parsed markdown sections. Brief definitions:

Check	Definition (operational)	Modes
structural-completeness	Required headings: Executive Summary, Findings, Final Risk Overview.	all
evidence-grounding	Each parsed finding has non-empty evidence field.	all
consistency	Heuristic consistency of severity vs issue text vs recommendation.	all
empty-generic-detection	Report length and generic-phrase filter (short or boilerplate ⇒ fail).	all
control-grounding	Control IDs cited in findings must match retrieved control IDs (regex on finding control field).	RAG only

Headline score = (# checks passed) / (# checks in that mode). Full pass requires 100% of checks in that mode.

1.3 Models (measured confounder)

Configuration	Provider / model (from run metadata)
① OpenCode + RAG	opencode / qwen3.6-plus-free
② GPT direct	openai / gpt-5.4-mini

Confounder: ② uses a different foundation model than ①/③ in this benchmark export. Any cross-suite difference partly reflects model capability and tokenizer behavior, not only retrieval vs non-retrieval.

2. Input scale and context (observed)

Directory ingest for RAG uses `DEFAULT_MAX_CHARS = 120000` (`packages/opencode/src/security/schema.ts`). When cumulative file text exceeds this, ingestion truncates (`clipped: true` in metadata).

Metric	① OpenCode + RAG	② GPT direct	③ OpenCode baseline
Runs with metadata	50	50	—
Mean <code>files</code> count (metadata)	5530	1.0	—
Runs with <code>clipped: true</code>	50 / 50	2 / 50	—
Mean <code>input.txt</code> size (bytes)	120,477	7,507	—

2.1 Context window vs this benchmark

② **GPT direct is not “winning because it hits the context window.”** In these runs, ② ingests *one file per target*; observed mean `input.txt` size is orders of magnitude smaller than ①’s aggregated repo slice. Under-filling context reduces truncation pressure and usually stabilizes formatting. ① routinely sets `clipped: true` and presents tens of thousands of files scanned in metadata — the model sees a partial, concatenated corpus, which increases failure rates on structure and evidence parsers independent of “running out of tokens” in the API sense.

Why ~0.905 is not “94%.” The value 0.905 is a mean on [0,1] over four boolean checks per run (headline for ②). Equivalently, ~90.5% of those check outcomes are passes in expectation (~3.62 of 4 per run on average). It is not an accuracy against ground-truth vulnerabilities.

3. Results — ① OpenCode + RAG (primary)

This section summarizes the **primary** experimental arm: OpenCode in RAG mode over bench50 repository roots, with ISO control retrieval and five verification predicates per run. Every numeric quantity below is derived from the same $n = 50$ rows as the primary results.json passed to this renderer. A “run” is one target repository processed end-to-end; “completed” means the pipeline produced a scored verification artifact without a top-level error.

We report two parallel summaries. The **headline** row uses all five checks and divides by five, so it is the quantity that Figure 1 histograms. The **shared-four** rows restrict to the checks that also exist in ② and ③; they are the only fair within-run contrasts against those baselines and match the upper bar in Figure 3. Control-grounding affects only the headline, not the shared-four column; a large gap between the two means therefore indicates that grounding passes or failures are materially moving the five-check average relative to the four-check slice—Figure 2’s control-grounding row gives the marginal pass rate.

Metric	Value
Completed	50 / 50
Errors	0
Headline mean (÷5 checks)	0.736
Median headline	0.800
σ (headline)	0.103
Full pass (5/5)	1 (2.0%)
Mean on four shared checks only	0.725
All four shared pass	12 (24.0%)

Reading the summary table. *Headline mean* is the arithmetic mean of per-run scores in [0,1] where each score counts how many of the five checks passed. Because each check is Boolean, individual run scores take only the values 0.0, 0.2, ..., 1.0; the mean is therefore a weighted average of those mass points, not a continuous latent variable. *Median headline* is robust to outliers: if it sits above the mean, the distribution has a left tail (some very low scores); if below, a right tail. σ (*headline*) quantifies spread in the same units as the mean; for bench50-sized n , a σ near 0.10 typically indicates that most mass sits on two adjacent discrete scores (here 0.60 and 0.80), which is exactly the pattern Figure 1 displays.

Full pass (5/5) is a stricter operational criterion than the mean: it counts runs where every predicate, including control-grounding, succeeded. A low full-pass rate with a moderate mean is common when one check is hard and the others are easy—readers should inspect Figure 2 to see which row is the bottleneck. *Mean on four shared checks* recomputes each run’s score using only structural-completeness, evidence-grounding, consistency, and empty-generic-detection, then divides by four; *All four shared pass* is the fraction of runs where all four of those passed, ignoring control-grounding. That pair is the correct bridge to §3.1 and Figure 3.

Discrete headline mass. On the five-check scale, score 0.60 means exactly three checks passed and two failed; 0.80 means four passed and one failed; 1.00 means a perfect sweep. In this dataset: 0.60 → **17** runs; 0.80 → **32**; 1.00 → **1**. Together, 49 of 50 completed runs (98.0% of successful runs) land on those two scores alone, which justifies interpreting the experiment as “usually one or two check failures per repo” rather than a smooth degradation of quality.

3.1 Reference runs — ② and ③ (headline ÷4)

Configurations ② and ③ use a **four-check** headline (no control-grounding). Their headline mean is therefore *not* comparable to ①’s headline mean without normalization; §4 and Figure 3 use the shared-four construction for that purpose. The table below still lists headline ÷4 for ②/③ because that is what those pipelines optimize locally, and because readers often want the raw benchmark export numbers side by side.

Configuration	n	Headline mean	Full pass	Mean (4 shared)	All 4 shared pass
② GPT direct	50	0.905	62.0%	0.905	62.0%
③ OpenCode baseline	— (pass opencode results.json)				

② GPT direct. The headline mean 0.905 is the average fraction of the *four* checks passed per run. It is numerically similar to the “Mean (4 shared)” column for ② because the shared-four set is exactly those four checks. **Full pass** is the percentage of runs with score 1.0 on that four-check scale—i.e. all structural, evidence, consistency, and generic-content predicates passed. This number must not be quoted as “accuracy” against ground truth; it only measures conformance to the automated rubric in verification.ts. The high headline mean under ② co-occurs with small single-file inputs and a different model than ① (§1.3), so it does not isolate the effect of “no RAG.”

③ OpenCode baseline missing from this PDF. Regenerate with the merged OpenCode bench50 JSON as the third argument, for example: `bun benchmark/render-benchmark-pdf.ts /home/aggerio/temp/opencode/outputs/benchmarks_rag/2026-04-07T18-12-56-936Z <path/to/gpt/results.json> outputs/benchmarks/merged-opencode-bench50.json` — or produce a fresh baseline with `./benchmark/run-opencode-baseline-only.sh benchmark/targets-50.txt` so ③ targets the same single files as ②. Without that file, Figure 3 and Table 1 omit the third column and cross-suite prose should avoid claiming baseline numbers.

4. Distributional summary (percentiles)

Percentiles describe where most of the probability mass sits without assuming a parametric family. We report them for *headline* scores (different denominators per configuration) and again for the *shared-four* sub-score (common [0,1] scale across ①–③).

Empirical percentiles are order statistics: p50 is the median; the interval [p25, p75] is a robust interquartile range. For discrete per-run scores, several percentiles may coincide (e.g. if more than half of runs share the same headline value).

Percentiles of *headline* verification score per configuration (empirical CDF, no parametric assumption).

Configuration	p10	p25	p50	p75	p90
① OpenCode + RAG (headline ÷ 5)	0.600	0.600	0.800	0.800	0.800
② GPT direct (headline ÷ 4)	0.750	0.750	1.000	1.000	1.000

Same for **shared-four** score (fraction of shared checks passed):

Configuration	p10	p25	p50	p75	p90
① OpenCode + RAG	0.500	0.500	0.750	0.750	1.000
② GPT direct	0.750	0.750	1.000	1.000	1.000

4.1 Reading the percentile tables

The headline rows use different scales: ① divides by five checks, ② and ③ by four. Therefore **do not compare p50 values across those rows as if they were the same random variable**. Use the *shared-four* block for cross-configuration shape comparisons on a common [0,1] score. For ①, the gap between p50 and p75 shows concentration: a tight IQR implies most runs fail the same small subset of checks (see Figure 2). For ②, high p90 with p50 near 1.0 indicates many perfect four-check runs and a long lower tail—consistent with occasional structural or generic-content failures.

If p10 for ① sits at or below 0.60 while p50 is 0.80, at least 10% of runs scored no better than a single-check failure on the five-check scale even though the “typical” run loses only one check—report both median and tail when writing a paper’s results paragraph. If shared-four p90 for ② is 1.0 but for ① is lower, the gap is consistent with harder inputs and/or the extra grounding constraint, not necessarily with worse security reasoning in an absolute sense.

5. Figures and captions

The following plots summarize the **primary** configuration (① OpenCode + RAG, $n = 50$). Figure 3 adds ② and ③ on a *harmonized* four-check sub-score so that bar heights are not artifacts of different denominators. When reproducing this section in a paper, we recommend placing Figure 1–3 in order and citing Table 1 for per-check detail.

5.1 Figure 1 — Headline score histogram (① only)

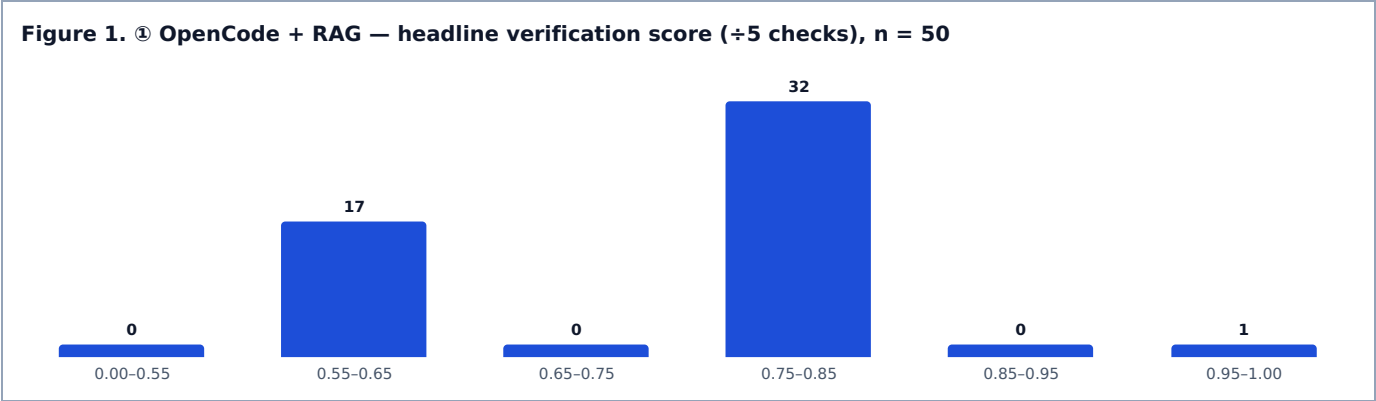


Figure 1 — interpretation. The horizontal axis partitions the *headline* verification score for ① into bins on $[0,1]$; the vertical bars count runs falling in each bin. Because the headline score is the fraction of **five** checks passed, only discrete values $\{0.0, 0.2, \dots, 1.0\}$ are mathematically possible; in practice this dataset clusters at **0.60** (17 runs) and **0.80** (32 runs), corresponding to exactly one or two failed checks among five. The modal bin is **0.75-0.85** (32 runs). The single-run mass at 1.00 (1 run) represents a rare case where all five checks, including control-grounding, pass. This histogram should be read as a *compression* of a five-dimensional boolean outcome into one scalar, not as a continuous latent “quality” measure.

5.2 Figure 2 — Check-wise pass rates (① only)

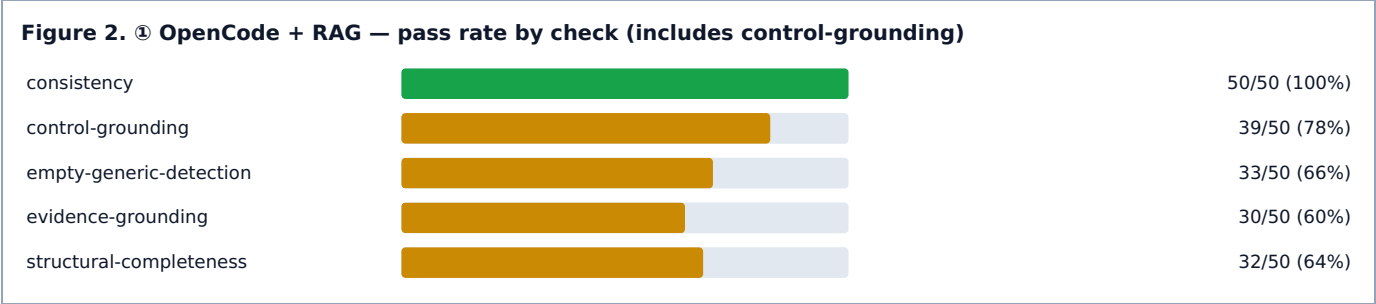


Figure 2 — interpretation. Each row names one automated check from the verifier; the shaded bar is the percentage of the 50 runs in which that check *passed*. The grey track is 100% width. The check with the **highest** pass rate here is **consistency** at 100% (50/50). The **lowest** pass rate is **evidence-grounding** at 60% (30/50); that bottleneck usually explains the 0.60 and 0.80 mass in Figure 1 (one or two failed checks on a five-check scale). **Control-grounding** (RAG-only) passes in 78% of runs; failures mean the model cited a control token that did not appear in the retrieved set for that run—either a hallucinated ID or a formatting mismatch with the parser’s regex. Readers should cross-reference Figure 2 with **Table 1**, which places the same percentages beside ② and ③ for shared checks.

5.3 Figure 3 — Shared-four comparison (①, ②, ③)

Figure 3. Shared-four comparison (fixed column order: ① → ② → ③)

Figure 3 uses the same four boolean checks for every configuration. Values are *not* percentages out of 100 unless labeled; mean 0.905 means on average 90.5% of those four checks passed per run (i.e. ~ 3.62 of 4).



Figure 3 — interpretation. Columns are fixed in evaluation order: ① **OpenCode + RAG**, then ② **GPT direct**. Within each column, the *taller* upper bar encodes the mean of the four checks that exist in every mode (structure, evidence, consistency, generic-content filter). The *lower* bar encodes the fraction of runs where **all four** of those checks pass simultaneously—a stricter criterion than the mean. Numerically, ① achieves a shared-four mean of **0.725** and an all-four pass rate of **24.0%**; ② yields **0.905** and **62.0%**. If ②'s mean exceeds ①'s, that reflects the confounders in §1.3 and §2 (model change, input cardinality, absence of a fifth check), not a proof that raw GPT is a superior security auditor. The scientific role of Figure 3 is to separate *template-level* agreement with the rubric from the *retrieval-grounding* obligation that only ① enforces (Figure 2, bottom rows / Table 1).

6. Synthesis for the paper text

A concise paragraph suitable for adaptation in a results section might read: “Under the OpenCode heuristic verifier, RAG over repository roots (50 runs) achieved a mean headline score of 0.736 on five checks (median 0.800, $\sigma = 0.103$). The empirical distribution (Figure 1) is concentrated at scores 0.60 and 0.80, consistent with one-two failed checks per run. Check-wise analysis (Figure 2) shows that evidence-grounding is the binding constraint (60% pass), while control-grounding succeeds in 78% of runs. On the four checks shared with non-RAG baselines (Figure 3), the mean shared-four score is 0.725 for RAG versus 0.905 for GPT direct, with the caveat that GPT uses a different model and smaller inputs (§1.3, §2).”

6.1 Threats to validity

- **Construct validity:** Scores measure agreement with parser rules, not real vulnerability density.
- **Internal validity:** Confounded comparison between ② and ①/③ via model family and prompt shape.
- **External validity:** Bench50 snapshots may not represent proprietary codebases or other languages.

6.2 Ordering convention (reporting)

- Configurations are labeled ① → ② → ③ throughout; this fixes reader expectation for an OpenCode-centric paper and does not imply ① dominates every numeric column.
- **Why ② can exceed ① on shared-four:** smaller non-clipped inputs; no fifth check; different model (§1.3); single-shot outputs often satisfy markdown structure detectors.
- **What ① adds:** measurable control-grounding (78% here) and per-run `retrieved_controls.json` for audit trails—absent in ②/③.
- **③ vs ②:** With aligned file lists, both use four checks; residual gaps reflect agent vs one-shot generation, not retrieval.

7. Table 1 — Check-level pass rates (all configurations)

Table 1 reports, for each check and each configuration, the percentage of runs in which that check's boolean predicate evaluated to true. Cells marked “—” indicate that the check is undefined for that configuration (e.g. control-grounding applies only to RAG). Percentages are row-comparable across columns for the same check name; they are *not* calibrated to sum to 100% across rows.

Verification check	① OpenCode + RAG pass rate %	② GPT direct pass rate %	③ OpenCode baseline pass rate %
structural-completeness	64	64	—
evidence-grounding	60	100	—
consistency	100	100	—
empty-generic-detection	66	98	—
control-grounding	78	—	—

Table 1. Percentage of runs in which each check passed. “—” = check not defined for that suite (control-grounding is RAG-only). Rows follow a fixed order so PDF engines do not permute columns.

Reading Table 1 after Figure 2. Figure 2 is the RAG-only projection of the first numeric column of Table 1. For shared checks, compare column ① to ② and ③ to see whether failures are systematic (e.g. structural gaps in RAG but not in GPT) or model-specific. Large gaps on evidence-grounding or structural-completeness between ① and ② are consistent with truncated multi-file context in ① overwhelming the parser’s expectation of well-formed per-finding blocks. If ③ is available, it helps isolate whether those gaps come from the *agent* loop (③ vs ②) versus *retrieval + grounding* (① vs ③).

8. Limitations & reproducibility

This report is an **instrumentation snapshot**: it measures how often model outputs satisfy syntactic and light-semantic rules in a fixed verifier, not whether the underlying repositories are secure. A run can score 1.0 on all checks while missing real vulnerabilities, or fail structural checks while containing useful narrative. Readers should treat every percentage as *rubric pass rate*, not labeled precision/recall against a CVE oracle.

Verifier limitations. Structural-completeness keys off markdown headings and section boundaries; alternate report templates can fail despite coherent content. Evidence-grounding only tests non-emptiness of a parsed evidence field, not factual correctness of the cited lines. Consistency is a short heuristic over severity strings and recommendation text. Empty-generic-detection is a length and phrase filter—legitimate terse reports can be penalized. Control-grounding enforces alignment between cited ISO control tokens and the retrieval set returned for that run; it does not prove that the cited control is the best match for the finding.

Benchmark corpus. Bench50 is a convenience sample of public snapshots; it may over- or under-represent languages, build systems, and risk profiles relative to an enterprise monorepo. Cloning dates, default branches, and omitted secrets mean the same target string can yield different effective inputs over time. Pinecone index contents and embedding model versions should be pinned in any formal study; this PDF does not embed API revision metadata beyond what each run recorded.

Reproducibility. Regenerate this PDF with `bun benchmark/render-benchmark-pdf.ts <rag_dir> [gpt.json] [opencode.json]`. Align ③ with ②’s file list using `./benchmark/run-opencode-baseline-only.sh benchmark/targets-50.txt`. Archive `results.json`, `per-run verification.json`, `metadata.json`, `input.txt`, and `retrieved_controls.json` (for RAG) so third parties can recompute tables without re-invoking paid APIs. Document model IDs and temperature settings from each run folder when citing numbers in a camera-ready paper.